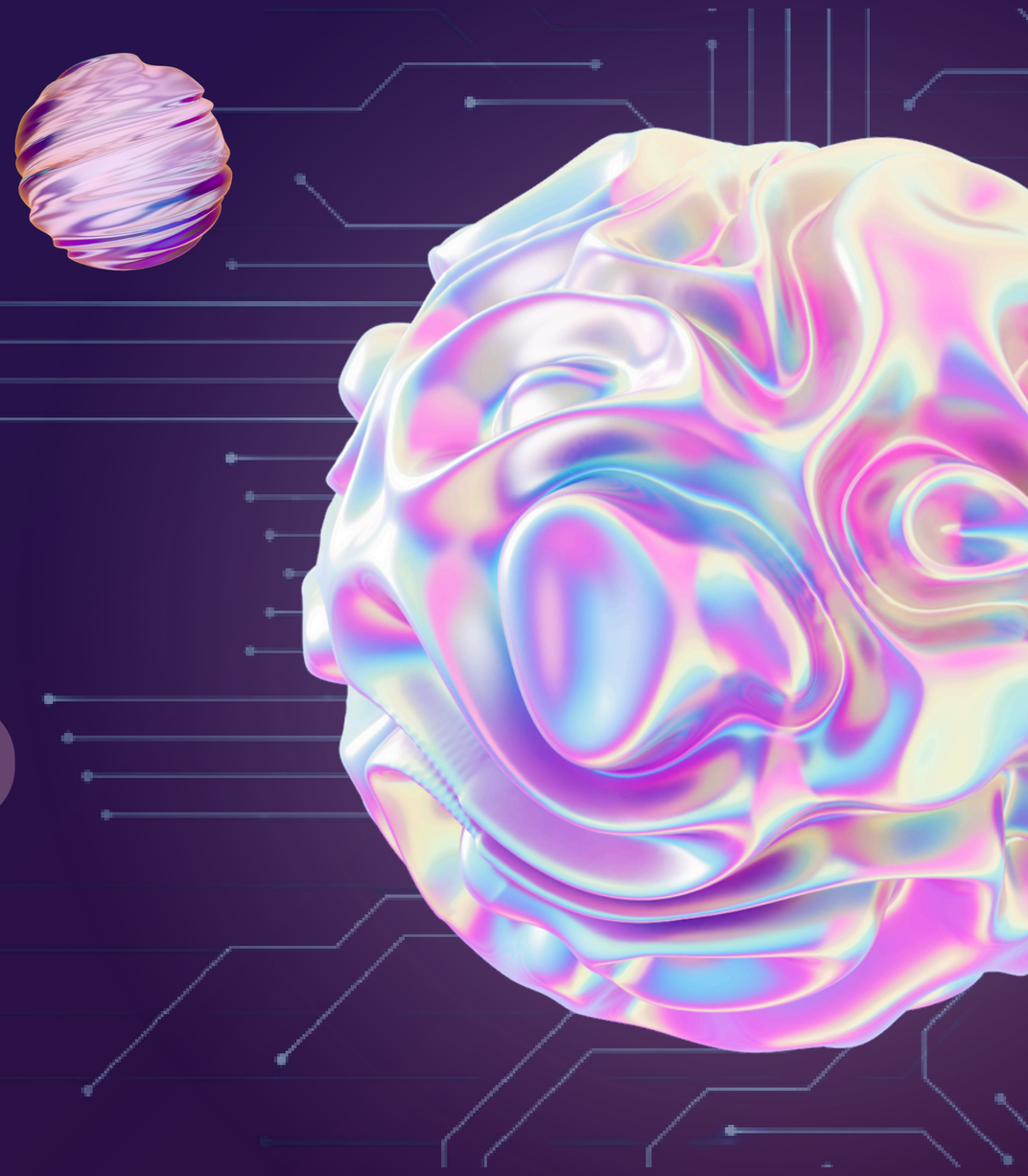
NUS SDS AY26/27

See, Detect, Understand

Introduction to Computer Vision

21/10/2026 (Wednesday)



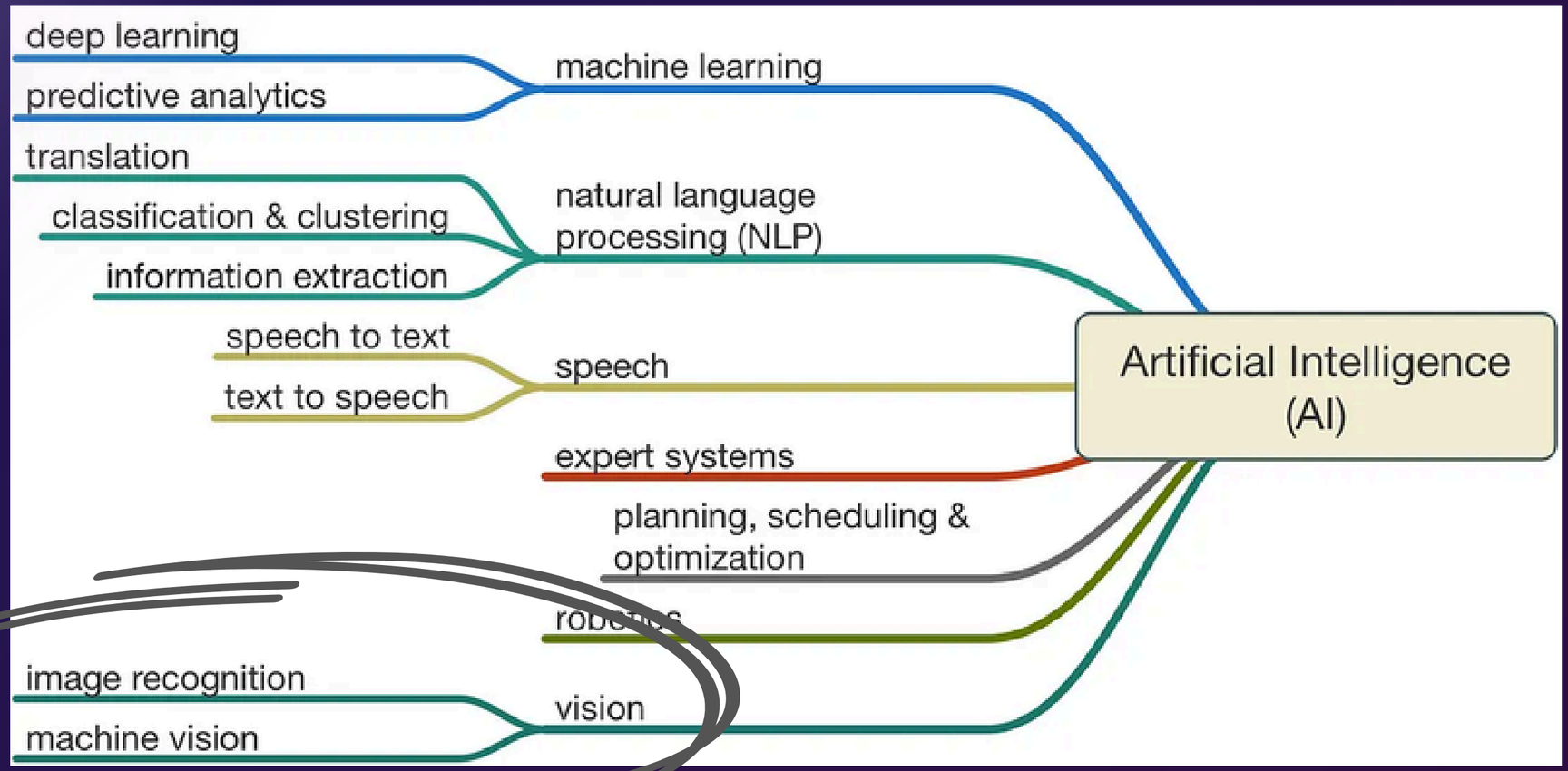
Team Members

- Ang Xuan
  Y4 DSA

- Brytania
  Y3 DSA

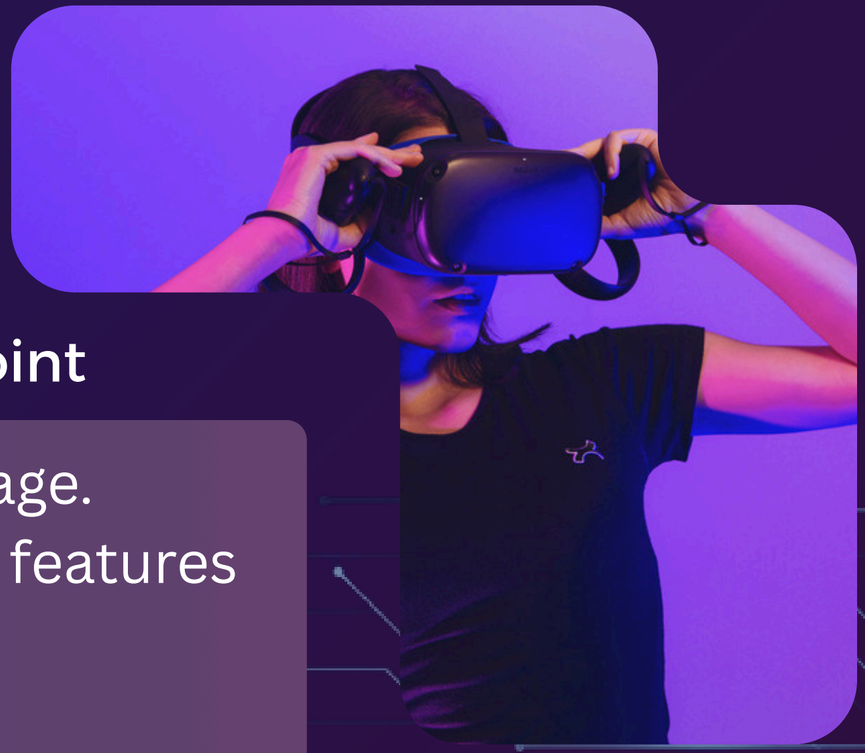
- Wei Tao
  Y2 DSA

Where are we right now?



Mindmap from https://medium.com/ml-ai-study-group/ai-mind-map-a70dafcf5a48

What is **Computer Vision?**



The Definition

A subfield of artificial intelligence (AI) that equips machines with the ability to process, analyze and interpret visual inputs such as images and videos. (IBM)

Our starting point

Represent an image. Understand how features are learned.

Demo: classify a photo with MobileNetV2 (a pretrained model)

Image classification

What is in this image?

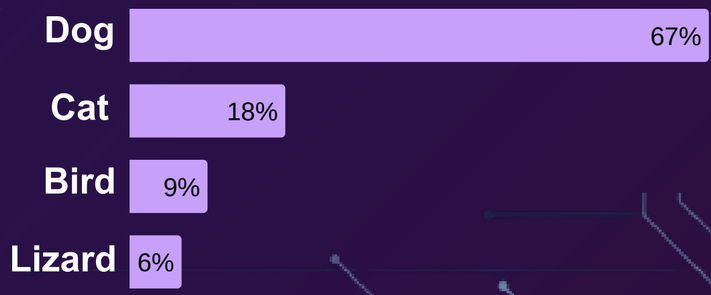
Input photo

Trained classifier

Suggested label

Dog

Score: 67%



Label: the suggested category.

Score: how strongly the model favours that category.

What we'll do in the notebook

Explore pixels

Read RGB values and image shape

Prepare the image

Resize, add a batch axis and scale values

Run the classifier

Use MobileNetV2 and read top predictions

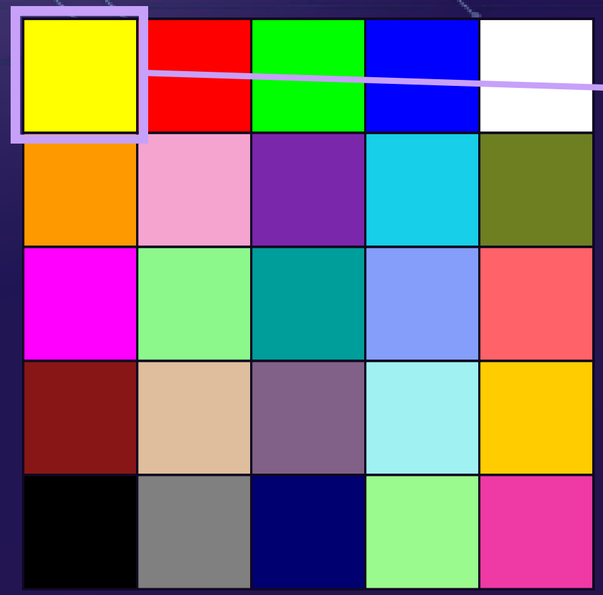
Test with blur

Compare labels and scores

Pixels and RGB channels

An image can be represented as a grid of numbers

An image is made of pixels



Each square is one pixel, the basic unit of a digital image.

RGB image: 3 channels

R G B

[255, 255, 0]

Red: 255

Green: 255

Blue: 0

Together, these values produce yellow.

Grayscale image: 1 channel

128

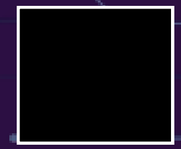
One brightness value per pixel.

Red
[255, 0, 0]

Green
[0, 255, 0]

Blue
[0, 0, 255]

Yellow
[255, 255, 0]

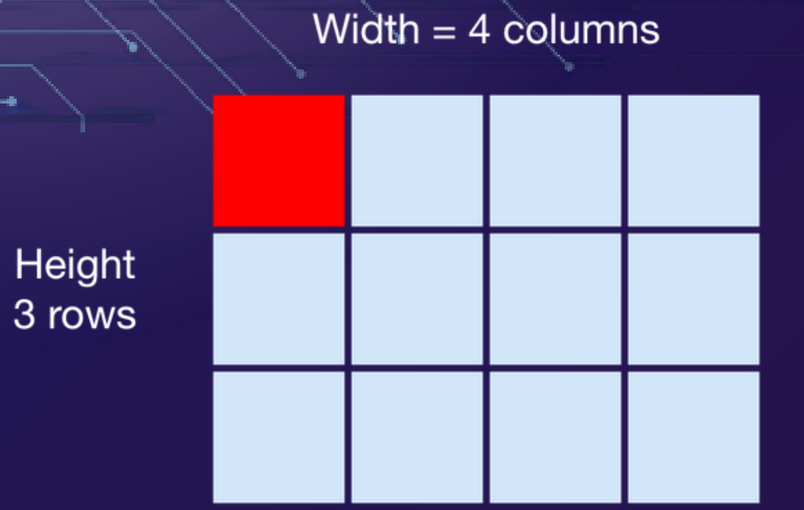


Black
[0, 0, 0]

White
[255, 255, 255]

Image dimensions and batches

One RGB image



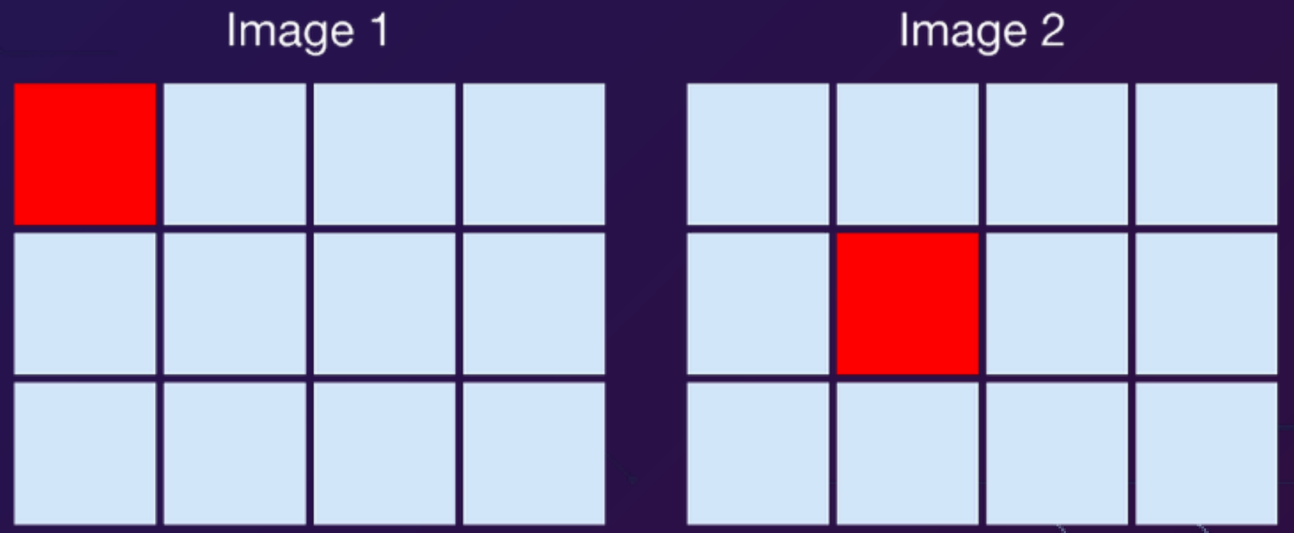
Highlighted pixel: 3 channel values

R: 255 G: 0 B: 0

(3, 4, 3)

Height Width Channels

A batch of two



2 images, each with shape (3, 4, 3)

(2, 3, 4, 3)

Images Height Width Channels

CNN classification overview

How a trained CNN model turns an image into a predicted label

CNN = Convolutional Neural Network

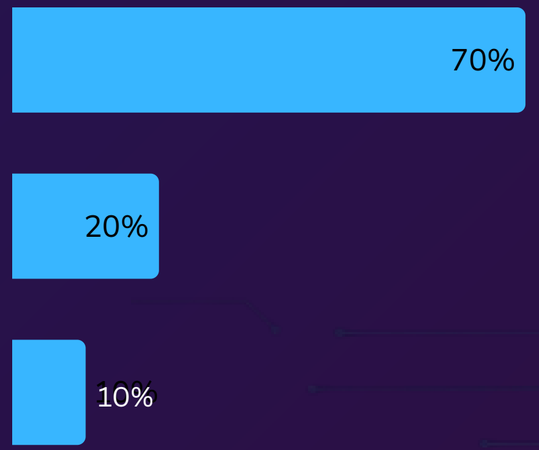
1. Input image

Pixel values represent the photograph.

2. Find patterns

Find edges, colours and textures, then combine the information.

3. Score categories



Illustrative three-class scores

Use the patterns to score the known categories.

4. Predicted label

Dog

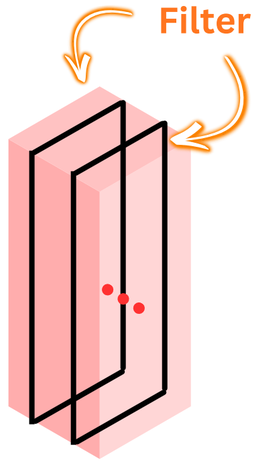
Select the category with the highest score.

Inside the trained CNN

For a new image, the trained CNN uses patterns it has already learned.

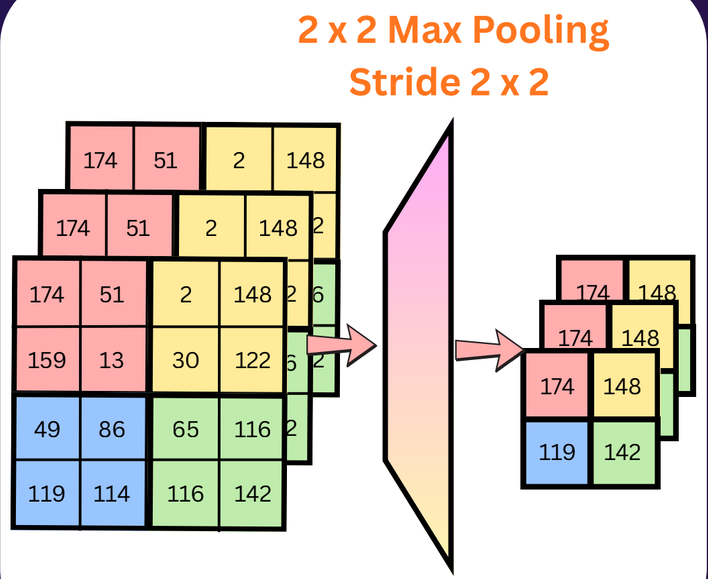
Types of CNN Layers

Convolutional Layer

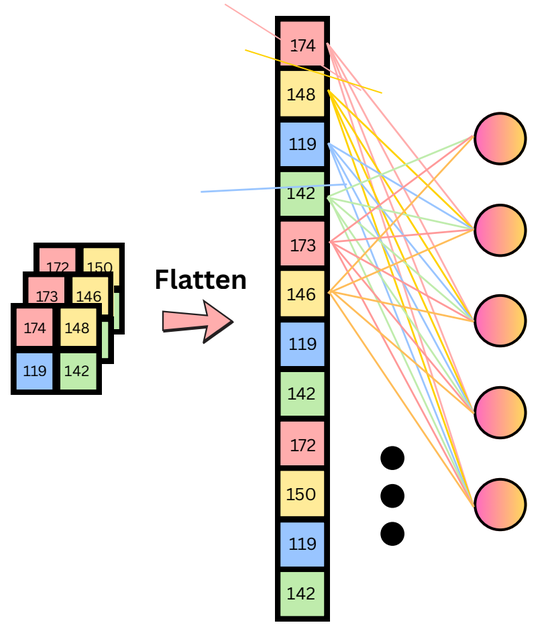


Each convolutional layer can have **different number of filters** which learn different feature.

Pooling Layer



Fully Connected Layer

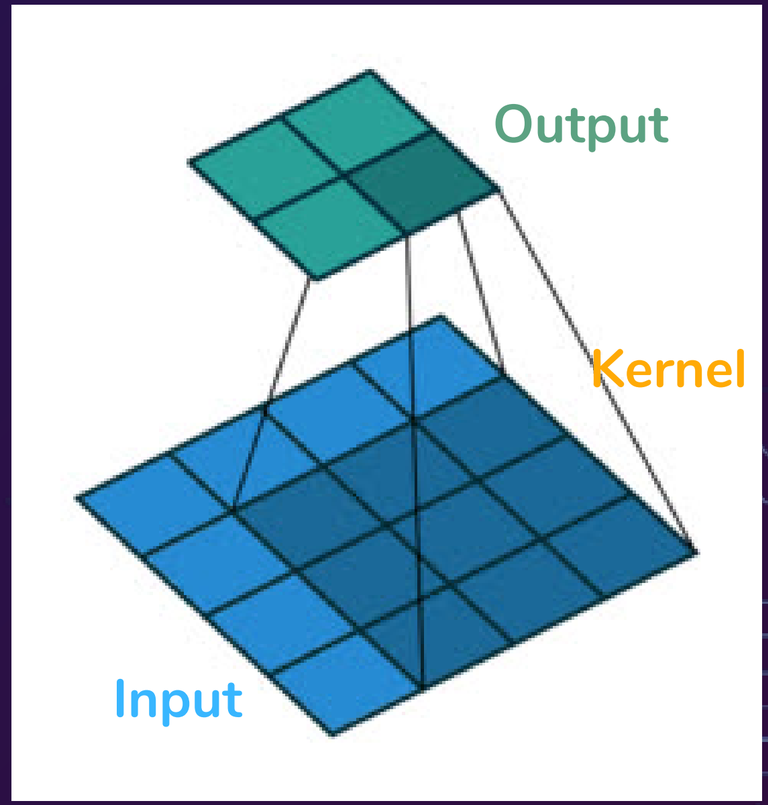


Convolution Operation

Procedure:

1. Place the **kernel** over a portion of the **input**
2. Multiply the elements of the **kernel** with the corresponding elements of the **input**, then sum them up
3. Repeat by sliding the **kernel** over the **input** image until the **kernel** has been placed over each **input** section.

Each element in the output matrix is a single number (sum of products)



Example Walkthrough

Input (6x6)

20	24	11	12	16	19
19	17	20	23	15	9
21	40	25	13	14	8
9	18	8	6	11	22
31	3	7	9	17	23
20	12	3	11	19	30

×

Kernel (3x3)

1	0	-1
2	0	-2
1	0	-1

→

Output (4x4)

3			

Sample calculation for first filter location

```
(20 x 1) + (24 x 0) + (11 x -1) +
(19 x 2) + (17 x 0) + (20 x -2) +
(21 x 1) + (40 x 0) + (25 x -1) = 3
```

What's Next?

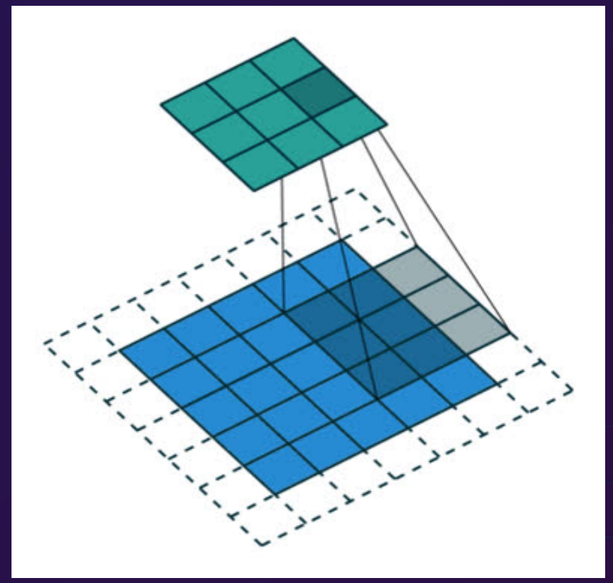
20	24	11	12	16	19
19	17	20	23	15	9
21	40	25	13	14	8
9	18	8	6	11	22
31	3	7	9	17	23
20	12	3	11	19	30

Stride to the next 3x3 patch and do the same thing until every portion is processed!

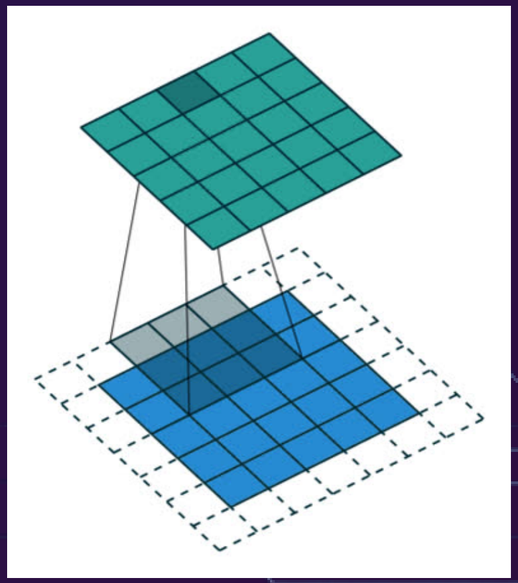
Striding

Stride is the number of pixels by which we slide our kernel matrix over the input. The operation happens from left-to-right, top-to-bottom across an image.

A smaller stride uses fewer positions, producing a larger feature map. Vice versa for a larger stride.



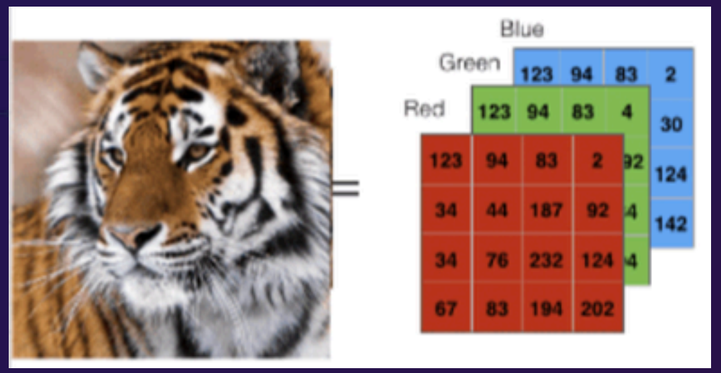
Stride = 2

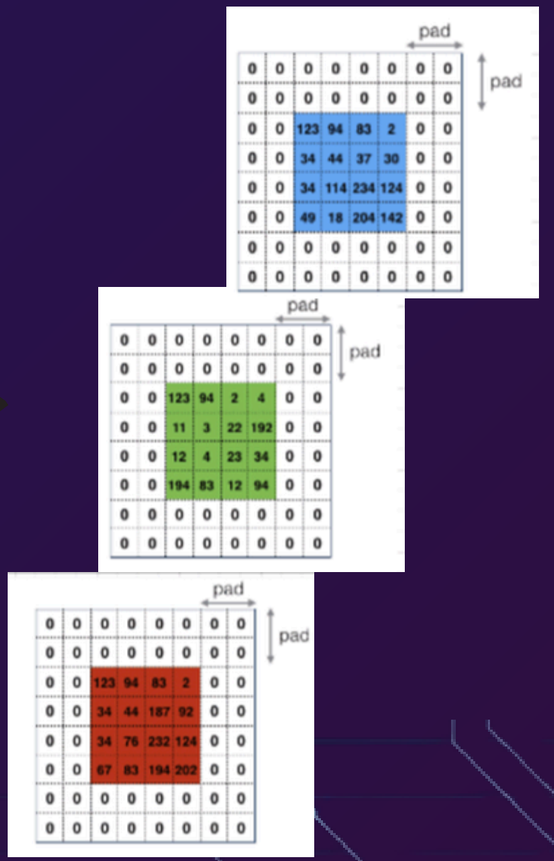


Stride = 1

Padding

The process of **adding extra pixels** (usually zeros) around the border of the images.





Why is it important?

As network becomes deeper, **convolution output will get smaller. Only** the **middle part** of the **image gets more focus,** and we lose information at the border.

Padding: zero and edge

Padding adds a border of values around an image or feature map before filtering.
This lets the filter cover the boundary and helps control the output size.

Zero padding

Fill the added border with zeros.

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

Original 4 × 4 input

Edge padding

Repeat the nearest boundary values.

1	1	2	3	4	4
1	1	2	3	4	4
5	5	6	7	8	8
9	9	10	11	12	12
13	13	14	15	16	16
13	13	14	15	16	16

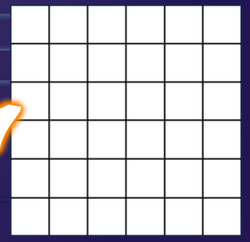
Added border (6 × 6 padded input)

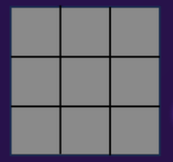
Both methods can give the same output size, but different filter responses near the boundary.

4 × 4 input, 3 × 3 filter, stride 1: output is 2 × 2 without padding, or 4 × 4 with a one-pixel border.

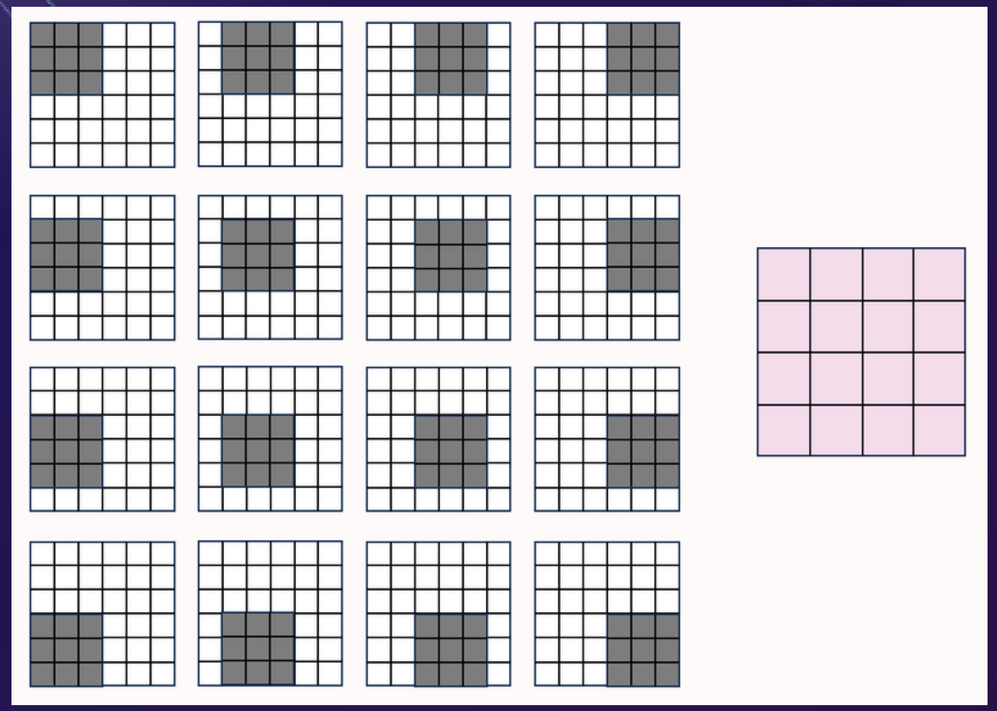
Our manual filtering helper uses edge padding.

Padding Example

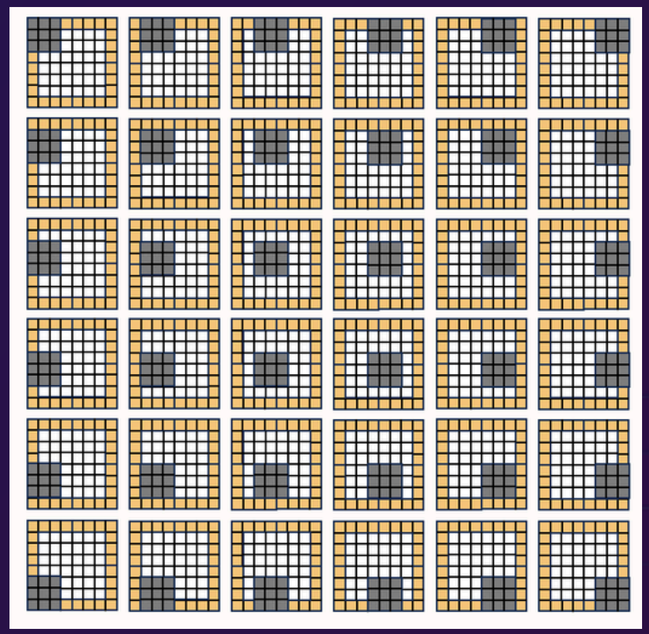




Let’s say our image has a 6x6 dimension, and the current Convolutional Layer is working on a 3x3 kernel.



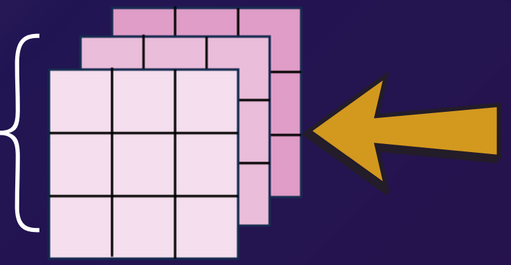
Without padding, we will get a 4x4 output after the convolution.

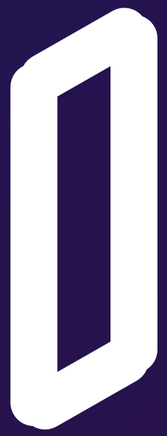


By adding one layer of padding to all sides, we will first get an 8x8 input. After the convolution (with stride = 1), we will get a 6x6 output, which has the same dimension as before the convolution.

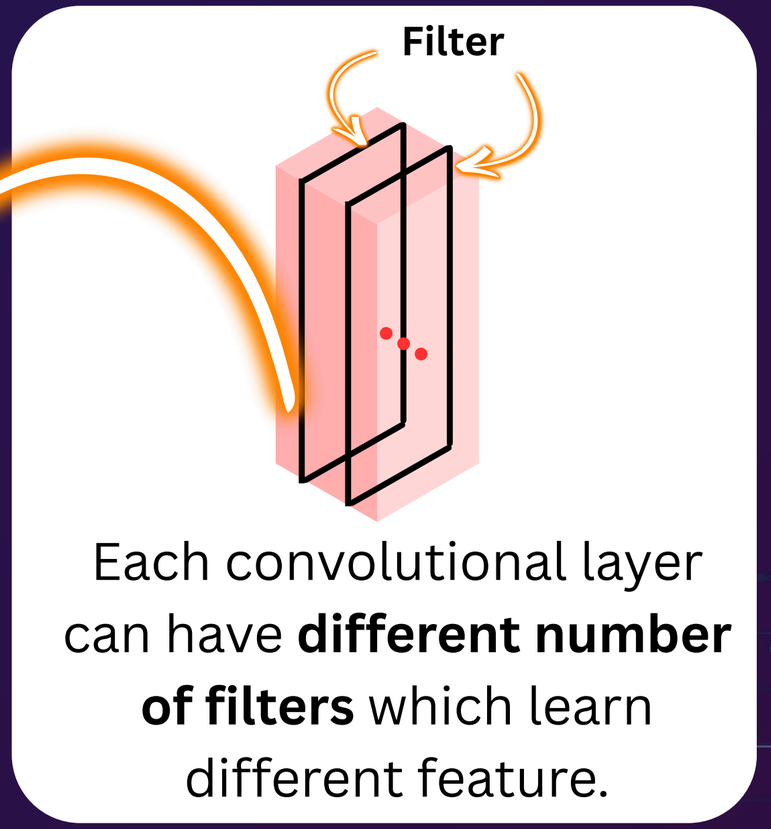
Convolutional Layer

Filter composed of C=3 kernels (C depends on the number of channels from the input image to this layer)





One of many filters in the convolutional layer

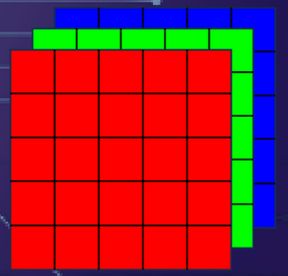


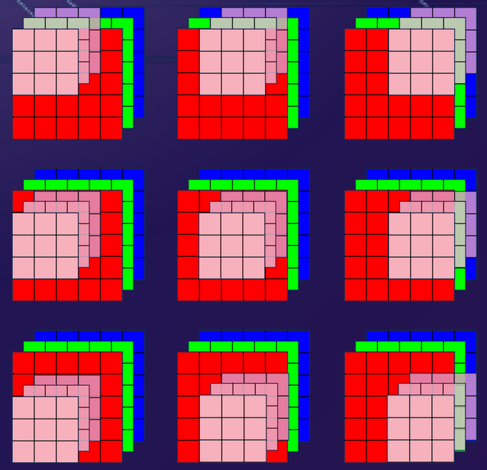
Each convolutional layer can have **different number of filters** which learn different feature.

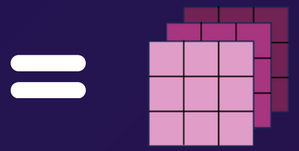
The output is of applying a filter is known as a 2D feature map. Convolutional layers typically contain many filters, meaning each convolutional layer produces multiple feature maps.

Note: the number of filters is a design choice!

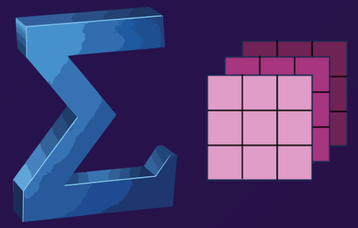
Walkthrough



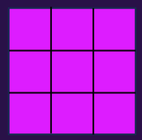




3D map

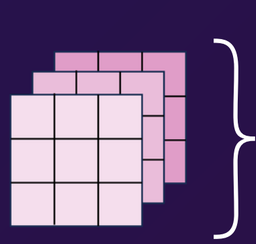


Pixel sum



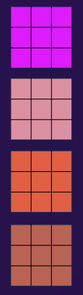
Feature map

Result from one filter only!

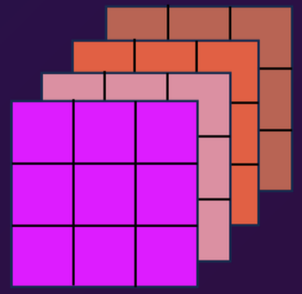


Filter composed of C=3 kernels

Say I have 4 filters in this convolutional layer



From Filter 1
From Filter 2
From Filter 3
From Filter 4



Activation functions and ReLU

ReLU is one type of activation function, commonly applied after convolution.

Definition: An activation function transforms a layer's computed responses into its output.

After convolution

-2	1	3
4	-1	0
-3	2	5

Example feature-map responses

Shape: 3 × 3

ReLU

Rectified Linear Unit

$$\mathrm{ReLU}(z) = \max(0, z)$$

$z < 0$: output 0

$z \geq 0$: keep z

z: one value in the feature map

Other nonlinear activations:
sigmoid and tanh.

After ReLU

0	1	3
4	0	0
0	2	5

Transformed responses

Shape: 3 × 3

ReLU changes response values while preserving the feature map's height, width and channels.

Nonlinear activations help CNNs capture more complex relationships in images.

Pooling Layer

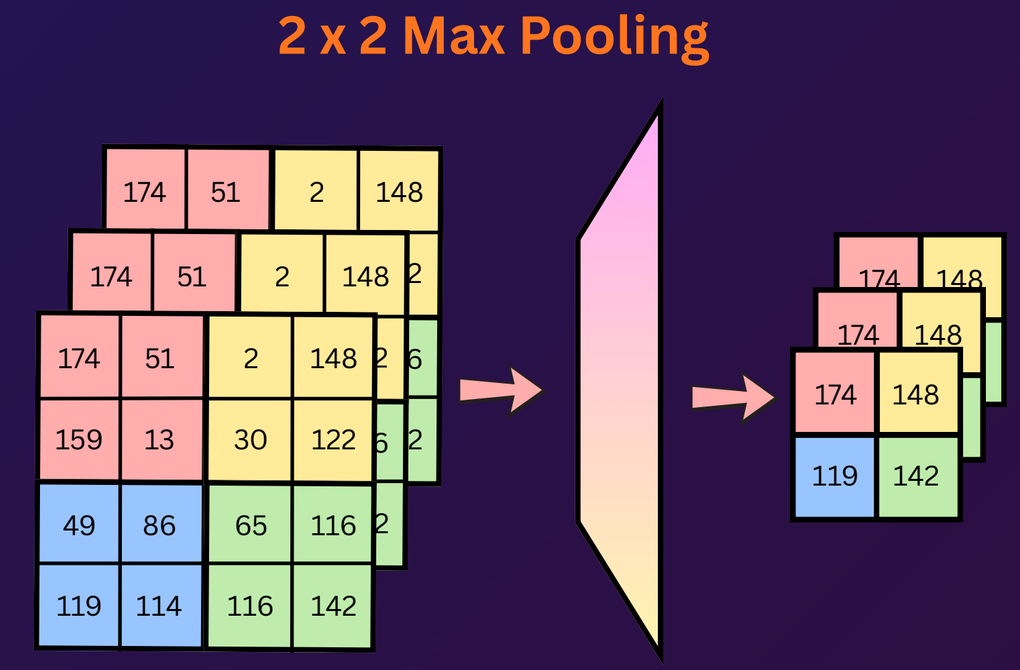
Purpose

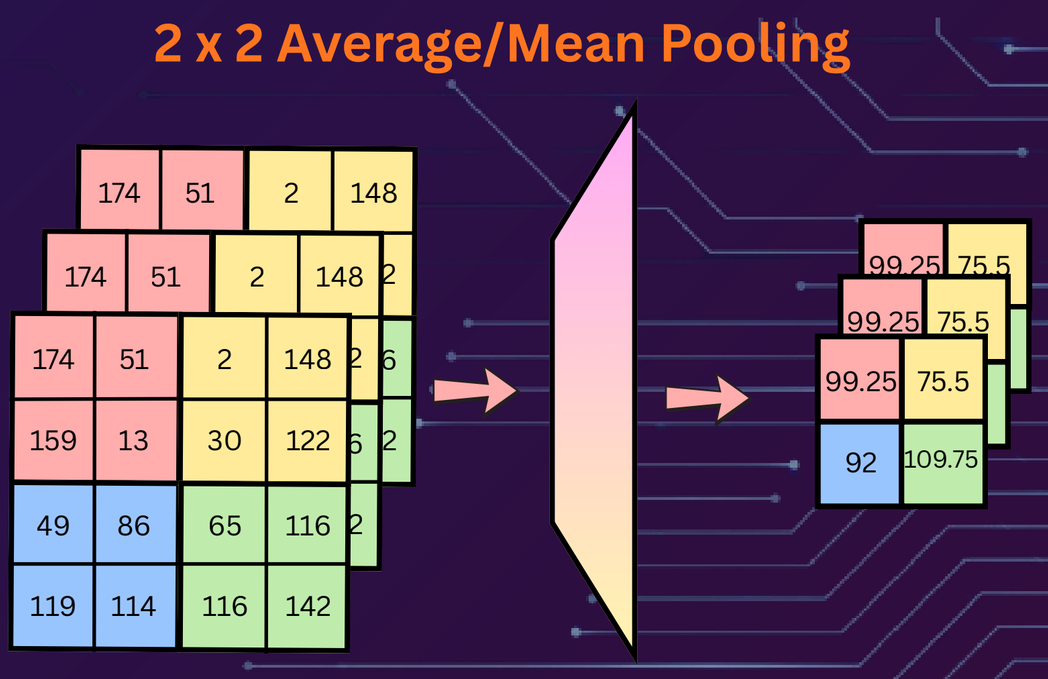
Reduce spatial dimensions

Pooling summarises nearby values to reduce the height and width of feature maps. This gives later layers fewer positions to process, reducing computational cost and the memory needed to sto**re feature maps.**

Recognise patterns even when they shift slightly

Max pooling keeps the highest value in each small area, helping the CNN recognise a pattern even when its position shifts slightly within that area.

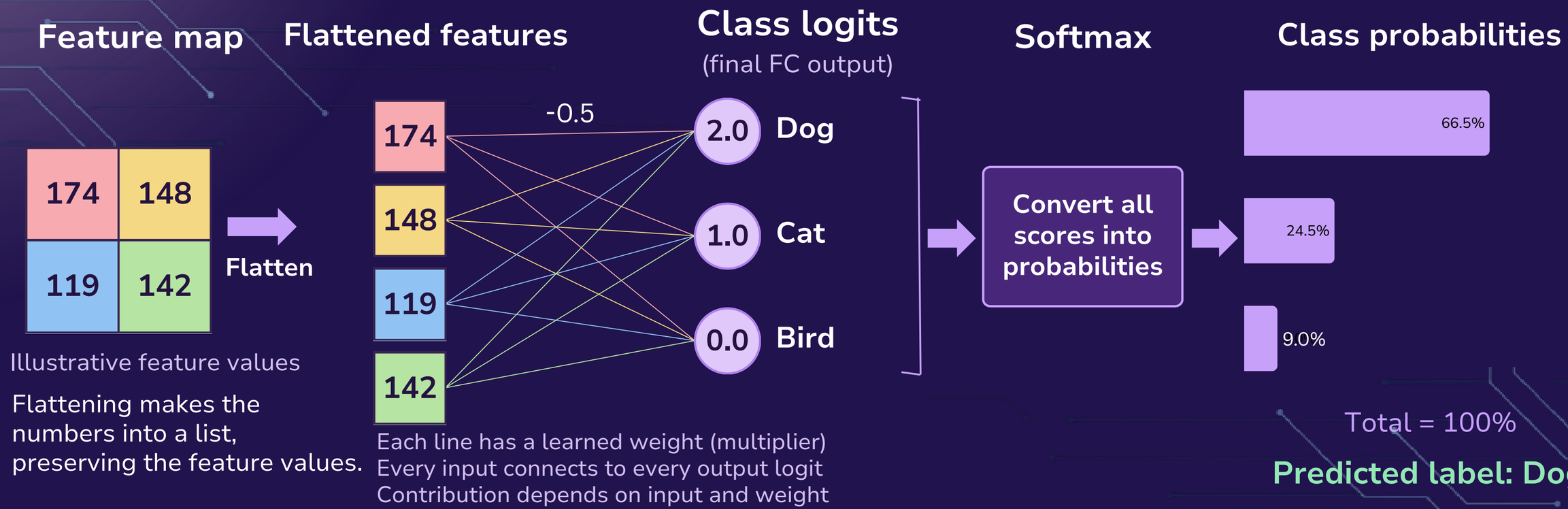




Fully connected (FC) layer and softmax

The final FC layer produces raw class scores. Softmax turns them into probabilities.

Each class score combines all input features using learned weights and a bias.



Logits: raw class scores

Can be negative or greater than 1.
Do not need to sum to 1.

Illustrative three-class example

Softmax: an activation function

Converts all logits into probabilities.
The probabilities sum to 1 (100%).

Training and inference

How a CNN learns from labelled examples and makes predictions

Training

Start with a CNN, then learn from labelled examples.

1 Labelled batch

True label: dog

True label: cat

Illustrative two-class example

2 Predict

Dog image: p(dog) = 0.20

Cat image: p(cat) = 0.80

3 Compute loss

Cross-entropy

Loss = −ln p(correct class)

Image	Loss
Dog	1.61
Cat	0.22

Batch loss ≈ 0.92

4 Backpropagate

Work backwards from the loss to guide the parameter update.

5 Update parameters

Labels

Repeat with the next batch

1 epoch = one complete pass through the training dataset

Batch 1 → **Batch 2** → **...** → **Batch N**

All training batches

Repeat over multiple epochs

Inference

Input image

Training or unseen image

Trained CNN

Parameters stay fixed

Class label + probabilities

Dog	0.87
Cat	0.13

Predicted label: dog

Our MobileNetV2 demo uses a pretrained model for inference.

Loss Functions

Measuring prediction error during training

Definition: A loss function compares a prediction with the correct label and returns a number that measures prediction error.

Correct label: Dog

A labelled training example

Two illustrative predictions for the same image

Model output	Prediction A	Prediction B
Dog (correct class)	**20%**	**80%**
Cat	80%	20%
Cross-entropy loss	**$-\ln(0.20) \approx 1.61$**	**$-\ln(0.80) \approx 0.22$**

More probability for the correct class gives a lower loss.

Loss = $-\ln(p)$ p = probability assigned to the correct class

ln is the natural logarithm

Some other types of loss functions

1. Mean Squared Error (MSE)
2. Mean Absolute Error (MAE)
3. Binary Cross-Entropy

Using a pretrained CNN

ImageNet is the dataset. MobileNetV2 is the model.

ImageNet

A labelled image dataset

ImageNet-1K: 1,000 categories

Pretraining

MobileNetV2

A CNN with pretrained parameters

weights="imagenet"

Loads the pretrained parameters.

During prediction

RGB image
(224, 224, 3)

→ **CNN feature extraction + pooling**

→ **Classifier**
1,000 raw scores

→ **Softmax**
1,000 probabilities

Our demo uses the pretrained parameters for inference.

Preparing an image for prediction

Use the same order as the notebook exercises.

1 Load RGB

(H, W, 3)

Original image size

2 Resize + float32

224 × 224

float32 values

(224, 224, 3)

Resize + np.asarray

3 Add batch axis

Batch size = 1

(1, 224, 224, 3)

np.expand_dims

4 Scale values

[0, 255] to [−1, 1]

v / 127.5 − 1

MobileNetV2 scaling

(1, 224, 224, 3)

preprocess_input

5 Predict

1,000 probabilities

(1, 1000)

model.predict

Different photo sizes work because we resize every image to 224 × 224 first.

The configured model receives (N, 224, 224, 3). N is the number of images.

Understanding Prediction Scores

Softmax assigns probability across the model's 1,000 known classes.

Inside the pretrained model

Image → **CNN features + pooling** → **1,000 raw class scores** → **Softmax** → **Class probabilities sum = 1 (100%)**

Illustrative example

Input image

Top 3: 95% Other 997 classes: 5%

Only 1,000 possible answers

Every predicted label comes from the 1,000 ImageNet classes.

No "none of the above" option

Unfamiliar inputs still receive scores for familiar labels.

A high score can still be wrong

Softmax allocates all 100% to known classes, even on unfamiliar inputs. The demo has no trained mechanism to reject unknown inputs.

A 90% score does not mean 90% test accuracy.

Prediction score: The score the model assigns to a category for this image.
Test accuracy: among all (unseen) new images, how many images the model classifies correctly

Testing Predictions with Blur

How does image detail affect the model's suggestion?

Resize to 224 × 224 → Apply blur → Batch + preprocessing → Predict

Illustrative blur levels

Original

Notebook radius: 0

Slight blur

Notebook radius: 4

Heavy blur

Notebook radius: 12

Run and compare

Check the top-1 label and its softmax score in the notebook.
The label may stay the same or change. Scores can rise or fall.

Only the input changes. The model parameters stay fixed.

Order matters

A blur radius is measured in pixels.
Applying the same radius before resizing can produce a different amount of blur.
Our demo: resize first, then blur. (the reverse will have different effects)

Classification tells us WHAT.
What else can CV tell us?

Classification
What is it?

Object Detection
What is it and where is it?

Segmentation
Which pixels belong to it?

Keypoint Detection
Where are the important points?

From Classification to Object Detection

Classification tells us **what** is in the image
Object detection tells us **what** is in the image AND **where**

Classification

- One overall prediction
- Predicts a label for the overall image (single class label)

Example output (softmax probabilities)

Dog	0.48
Cat	0.01
Person	0.22
Basketball	0.03
Football	0.10
Other classes	0.16

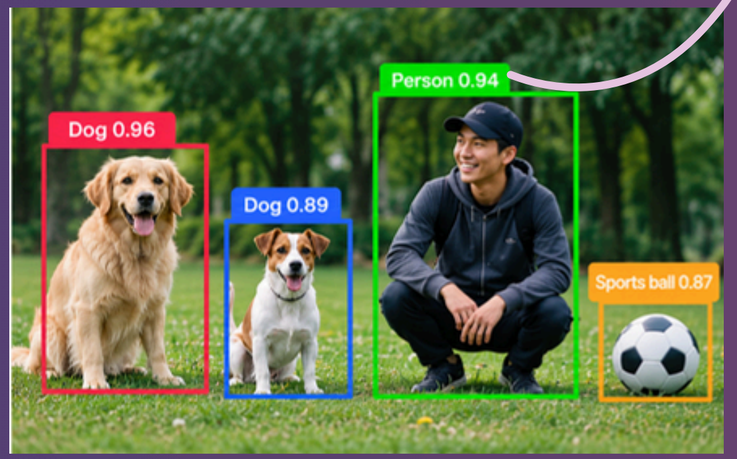
Highest probability: Dog (0.48)
→ predicted class: Dog

WHAT

WHAT +
WHERE

Object Detection

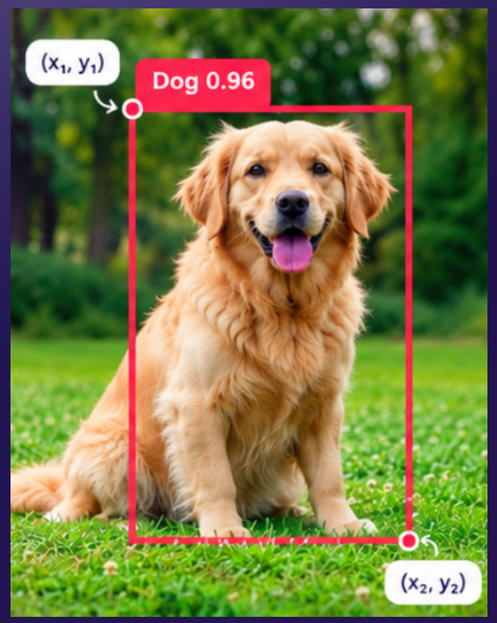
what does this number mean?



- Locate individual objects
- Detects multiple objects in the image
- Each detection has a class label, location, & confidence score

Understanding an Object Detection

Each detection gives us a **class label**, a **bounding box**, and a **confidence score**



Class Label

"what is it?"

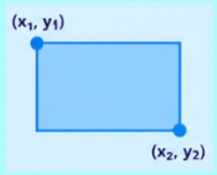
Dog

- Predicted category of the object
- Comes from a set of predefined classes (e.g. dog, cat, person, car, etc)

Bounding Box

Corner format

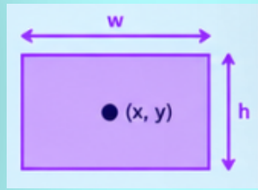
(x1, y1, x2, y2)



Top-left and bottom-right corners

Width-height format

(x, y, w, h)



Centre coordinates, width and height

As there are different ways to represent a bounding box → need to check the library documentation to confirm the format

Confidence Score

"how strong is the prediction?"

0.96

Confidence = objectness x class probability

- **Objectness** = how likely the box contains any object
- **Class probability** = how likely it belongs to a specific class
- **Higher score = stronger support** from the model
- The score is not calibrated (the model can be overconfident/ underconfident)
- Often used with a threshold to filter out weak detections (e.g. keep detections with score >= 0.5)

Object detection = class label + bounding box + confidence score

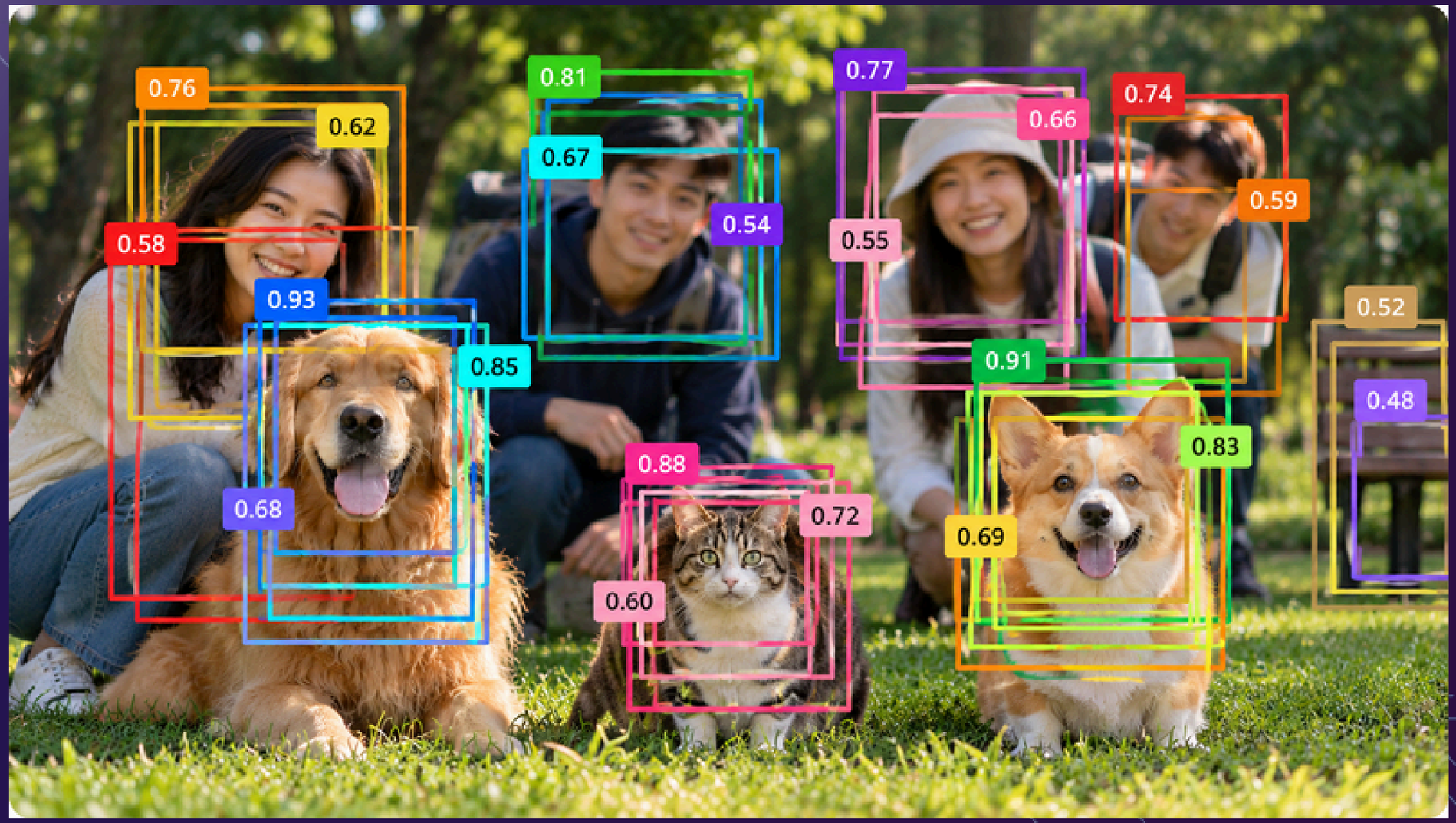
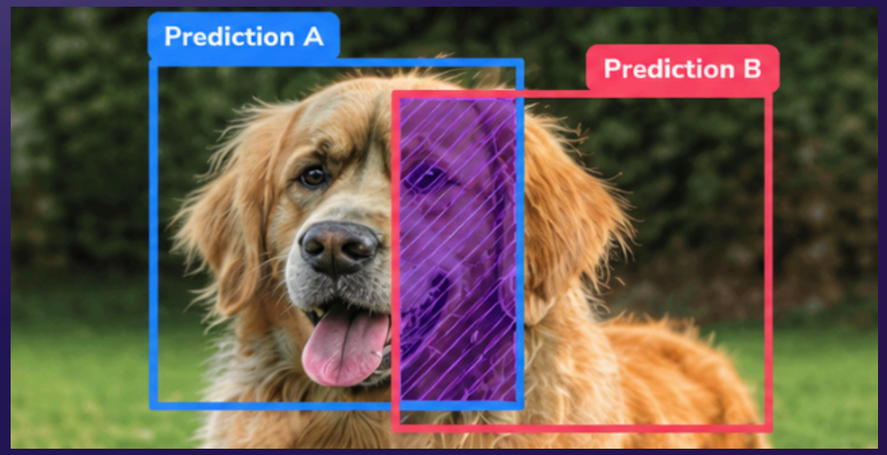


Image is generated by AI

How Do We Measure Overlap?

An object detector may produce multiple candidate boxes for the same object

1. Evaluate detection accuracy → compare the predicted box vs ground truth box (e.g. IoU >= 0.5)
2. Remove duplicated prediction (NMS) → compare predicted box A vs box B to identify overlapping



Intersection over Union (IoU)

A measure of how much 2 bounding boxes overlap → [0,1]

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

Intersection = area covered by both boxes
Union = total area covered by either box

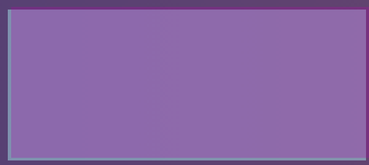
IoU = 0

No overlap

IoU = 0.5

Some overlap

IoU = 1



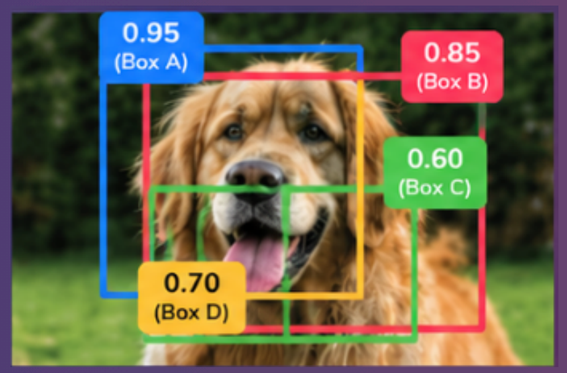
Identical (high overlap)

Non-Maximum Suppression (NMS)

Used to remove multiple overlapping boxes for the same object and keep the most confident one.
Works best for multi object image.

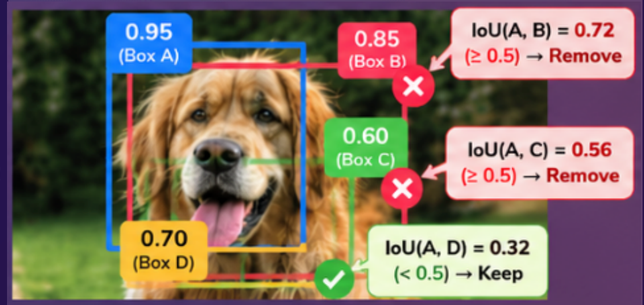
Our Iou Threshold = 0.5 → remove boxes with Iou >= 0.5 against the kept box

1) Multiple detections (before NMS)



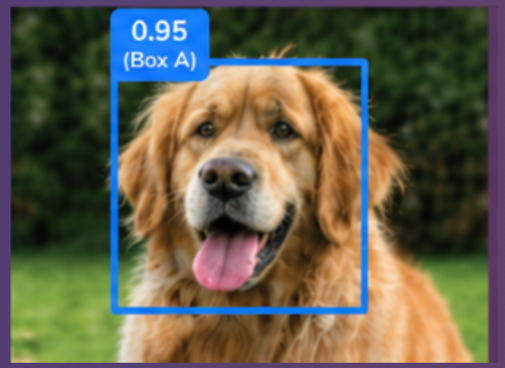
The model outputs multiple high-confidence boxes for the same object

2) Suppress overlapping boxes



Compare each remaining box with the highest-confidence box (Box A) and remove boxes with IoU >= 0.5

3) Final detection (after NMS)



Only the highest-confidence box is kept for the object

How NMS works

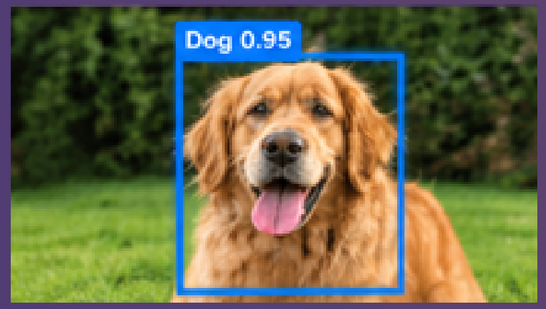
Sort all predicted boxes by confidence score (highest first) → Keep the highest confidence box → Remove boxes with Iou >= threshold against the kept box → Repeat for the remaining boxes until no highly overlapping boxes remain

NMS is class-specific (e.g. a dog and a person can have overlapping bounding boxes without one being removed

From Boxes to Pixels – Image Segmentation

Instead of just predicting a bounding box, image segmentation assign labels to individual pixels.

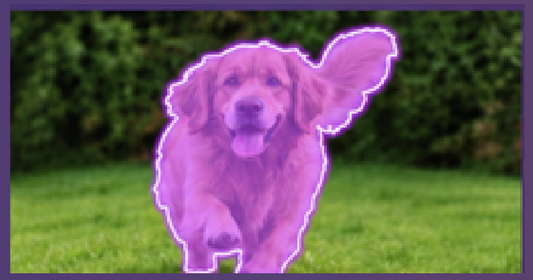
Object detection output



Bounding box

- Predicts a rectangular region containing an object
- Tells us where the object is

Image Segmentation



Pixel-level mask

- Predicts which pixels belong to an object or class
- Provides a more detailed (pixel-level) representation of the object's shape

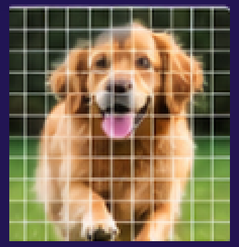
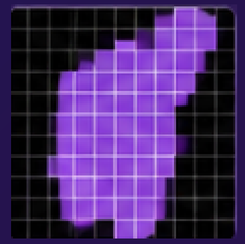


Image as pixels

An image is a grid of pixels

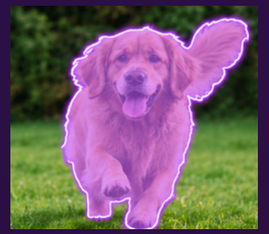


Model predicts a mask

The model outputs per pixel probability distribution

Apply threshold

Convert pixel probabilities to a binary mask



Final segmented mask

Only the pixels belonging to the object are kept → giving precise object shape

3 types:

Semantic Segmentation

Assign a class to each pixel

Instance Segmentation

Distiniguishes individual objects

Panoptic Segmentation

Semantic + Instance

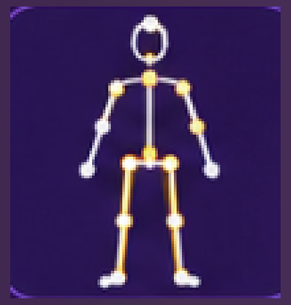
From Regions to Points – Keypoint Detection

Instead of detecting a whole region, keypoint detection predicts specific and meaningful points or landmarks on an object

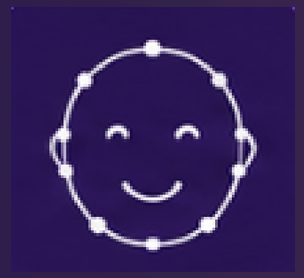
Keypoint detection: predicts a set of meaningful points or landmarks on the object

Why keypoints are useful?

Pose estimation



Gesture recognition



Facial landmark detection

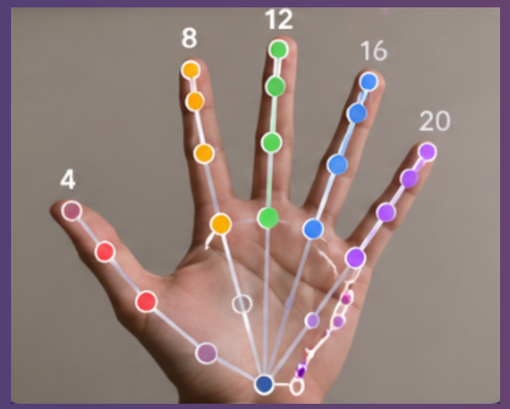
More compact and flexible

Example of pose estimation

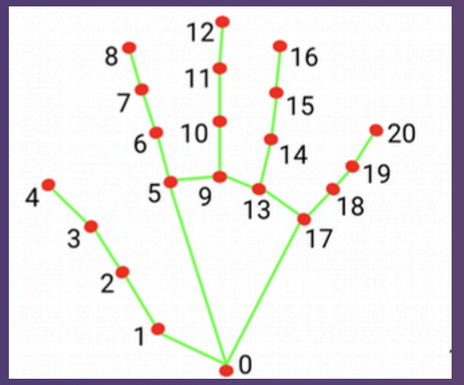
Keypoints and Hand Landmarks

Input image

MediaPipe hand landmarks



21 landmarks (0-20)

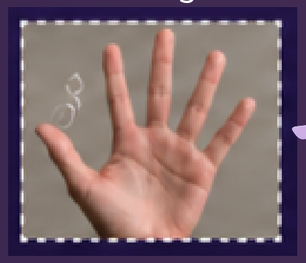


0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

Input image

Palm detection

Crop and align hand region



Predict 21 landmarks

Why detect & crop first?

- Focuses the model on the hand, reducing background noise
- Allows the landmark model to process a smaller and more relevant region

Each landmark has coordinates

For each of the 21 landmarks, MediaPipe outputs a 3D coordinate: **(x, y, z)**

- x, y: position in the image
- z: relative depth (how far from the camera)

Example of landmark coordinates:

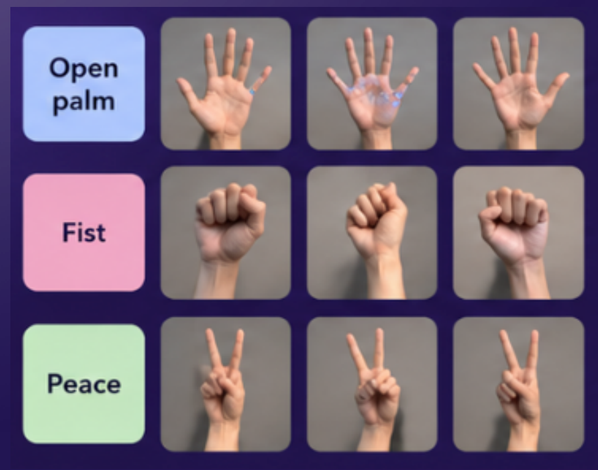
Landmark ID	x	y	z
0 (wrist)	0.52	0.78	-0.02
4 (thumb tip)	0.28	0.32	0.01
8 (index tip)	0.43	0.18	-0.03
...	...	...	...

x_0	y_0	z_0	x_1	y_1	z_1	...	x_{20}	y_{20}	z_{20}

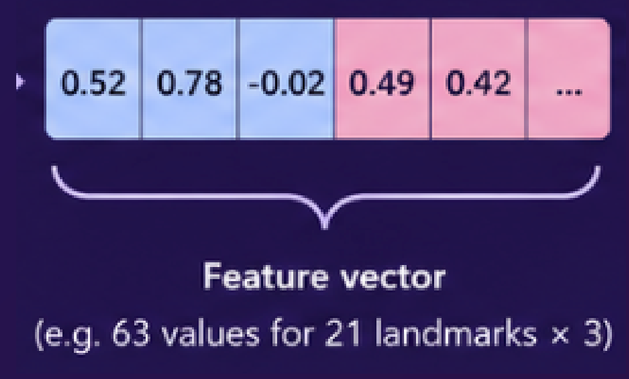
63 values total (21 landmarks × 3)

Hand Landmarks Dataset

1) Collect hand gesture data



2) Convert to numerical features



3) Final dataset

Feature 1	Feature 2	Feature 3	...	Label
0.52	0.78	-0.02	...	Open palm
0.50	0.81	-0.01	...	Open palm
0.21	0.33	0.03	...	Fist
0.24	0.31	0.02	...	Fist
0.60	0.18	-0.04	...	Peace
...	...	...	...	...

Dataset Summary

Rows
Each row = one hand example (an image)

Columns
Feature values extracted from hand landmarks (e.g. 63 features)

Label
The gesture class (e.g. open palm, fist, peace, like)

Hands-On: Can We Recognise the Gesture?

Let us build a classifier that can recognise gestures!

What are we going to do?

- Explore the hand-landmark representation
- Construct landmark features
- Train a gesture classifier
- Evaluate it on an unseen image
- Test the full pipeline on a new hand image

Pretrained CV

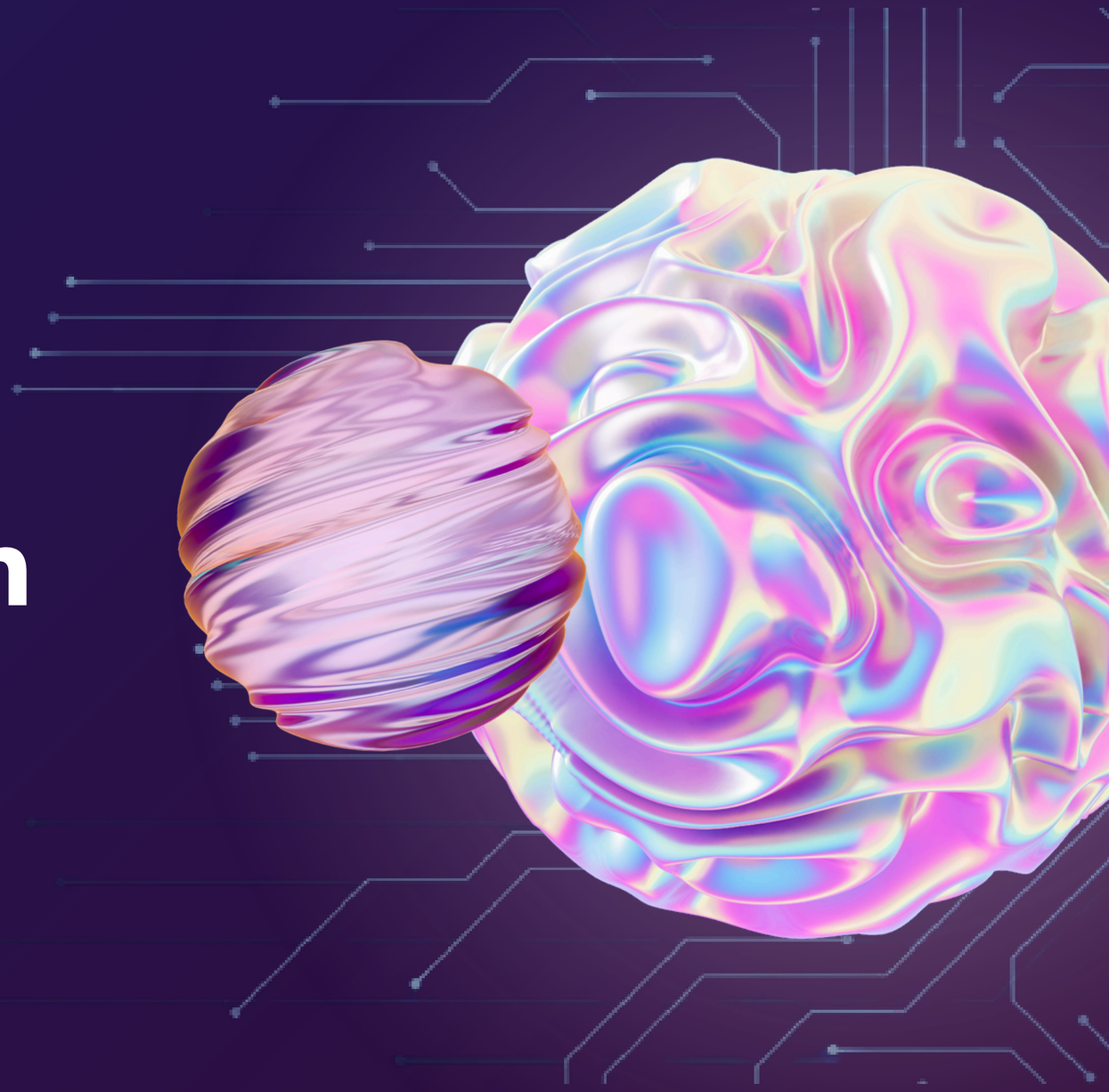
Image → MediaPipe → Landmarks

Hands-on

Features → Classifier → Gesture

Thank You
For Your Attention

See you in the next workshop!



NUS SDS AY26/27

Help Us Train Our Next Workshop! 🤖

Scan this QR code for attendance!

Feedback form!

Even AI models need feedback to improve, and so do we!

Scan the QR code to help us improve! 🚀

Follow our socials to stay in the loop!

SDS Instagram

NUS.SDS

SDS Channel

@NUSSDS

SDS Workshop AY26/27 Github

SDS Workshop AY25/26 Github

Thank you for coming today and see you in the next workshop!